Google Cloud IAM & Least-Privilege Governance (Roles, Service Accounts, Workload Identity)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Google Cloud IAM & Least-Privilege Governance 4

1. Solution Overview — Google Cloud IAM & Least-Privilege Governance 4

The Cloud IAM model — who can do what, deliberately 5

Reference design — the target end state 6

Deployment model — Assess → Design → Configure → Pilot → Validate → Expand 6

What this solution delivers 7

2. Requirements — Google Cloud IAM & Least-Privilege Governance 7

2.1 IAM administrative roles at the Organization node (not primitive Owner) 8

2.2 Google Groups as the source of truth for human access 8

2.3 Workload inventory and a federation plan for external callers 9

2.4 A named pilot folder or project for IAM changes 9

2.5 Required APIs enabled on the administration project 9

2.6 Application Default Credentials for the Python tools (no keys on disk) 10

Global Requirements — Google Cloud Platform (GCP) Foundation Solutions 10

1. A Google Cloud Organization linked to Cloud Identity / Workspace 11

2. Organization-level administrative access 11

3. A Cloud Billing account 11

4. The gcloud / IaC toolchain and API enablement 12

5. A resource-hierarchy, naming & environment design 12

6. Security, network & governance stakeholders and policy 12

3. Implementation — Google Cloud IAM & Least-Privilege Governance 13

Stage 1 — Assess (read-only baseline) 13

Stage 2 — Design (author on paper, apply nothing) 13

Stage 3 — Configure (apply to the pilot scope only, dry-run guardrails) 15

Stage 4 — Pilot (enforce on the pilot scope, observe) 15

Stage 5 — Validate (prove the reference design) 16

Stage 6 — Expand (roll out) and Rollback 16

5. Testing & Validation — Google Cloud IAM & Least-Privilege Governance 17

Test 1 — No primitive Owner/Editor except audited break-glass 17

Test 2 — Human bindings are group-based, not individual 17

Test 3 — New service-account keys cannot be created (guardrail enforced) 17

Test 4 — No exportable (user-managed) keys remain on migrated accounts 18

Test 5 — The pilot workload authenticates via Workload Identity Federation 18

Test 6 — Domain-Restricted Sharing blocks external principals 18

Test 7 — Reference-design assertion passes 18

Test 8 — Access-review report generates and reconciles . . . 18
6. Troubleshooting — Google Cloud IAM & Least-Privilege Governance 19
T1 — A team lost access after tightening bindings ("permission denied" for legitimate work) . . . 19
T2 — Org-policy guardrail is not blocking (a key still gets created) . . . 19
T3 — WIF workload cannot authenticate (STS token exchange fails) . . . 20
T4 — Access-review report is empty or missing Recommender flags . . . 20
T5 — Custom role rejects a permission ("permission is not supported") . . . 20
T6 — Domain-Restricted Sharing blocks a legitimate external principal . . . 21
T7 — "You need permission" when setting org policy or creating custom roles . . . 21
T8 — Locked out at the Organization node after an org-wide change . . . 21
7. Best Practices — Google Cloud IAM & Least-Privilege Governance 22
Roles . . . 22
Principals . . . 22
Kill exported keys . . . 22
Guardrails . . . 23
Review as a cadence . . . 23
Tooling discipline . . . 23
8. Deliverables Summary — Google Cloud IAM & Least-Privilege Governance 24
Documentation . . . 24
The 5 Python tools (`gcloud` / Cloud Client Libraries; read-only or `--dry-run` by default) . . . 24
Appendix A — PowerShell Scripts 26

Executive Summary — Google Cloud IAM & Least-Privilege Governance

The problem. On Google Cloud, over-granting is the default failure mode. The `Owner` primitive role is one click away, service-account **keys** can be exported and never expire, and access permissions accrete project by project with no one accountable for who holds what. The three most common findings in any Google Cloud security review are the same every time: **broad Owner/Editor grants, unmanaged exported service-account keys, and role sprawl.** Each is a standing breach risk — a leaked key or an over-privileged account is a direct path to the organization's cloud estate.

The outcome. This solution puts Google Cloud access under **least-privilege control**: role sprawl is tamed, exported keys are eliminated, and every binding is reviewed — so *who can do what* is **intentional and auditable** across the organization.

The approach. A repeatable Cloud IAM governance model, delivered as documentation plus five Python tools, established in six staged phases with explicit gates:

- **A least-privilege role model** — predefined roles first, custom roles only where nothing fits, and

no primitive Owner/Editor outside one audited break-glass path.

- **Group-based access** — humans are bound only through Google Groups at the correct level of the

resource hierarchy, so joiner/mover/leaver is a membership change, not an IAM edit.

- **Exported keys eliminated** — external and federated workloads move to **Workload Identity Federation**

(short-lived, auto-rotated credentials), and org policy makes new keys **impossible to create**.

- **IAM-restricting org-policy guardrails** — Domain-Restricted Sharing, key-creation disable, and

default-service-account grant disable make the wrong grant *impossible*, not merely discouraged.

- **A recurring access review** — IAM Recommender and Policy Analyzer drive quarterly re-attestation so

least privilege stays true over time.

Why it is safe and governed. Every change follows a **pilot-first** model — *Assess → Design → Configure → Pilot → Validate → Expand* — with pass/fail gates and a documented rollback. Guardrails are applied in **dry-run** and reviewed before enforcement; IAM policies are snapshotted for instant restore. The automation is **read-only or dry-run by default**, authenticates without any key file, and contains no secrets. Nothing reaches the Organization node until it has been proven on a contained pilot scope.

Bottom line. Google Cloud access under least-privilege control — role sprawl tamed, exported keys eliminated, bindings reviewed — intentional and auditable across the organization, proven on a pilot before you scale, at a fraction of the cost of a bespoke consulting engagement.

1. Solution Overview — Google Cloud IAM & Least-Privilege Governance

By default, Google Cloud makes it *easy* to over-grant. The `roles/owner` primitive role is one click away, service-account **keys** can be exported and emailed around, and bindings accrete at the project level with no one accountable for who holds what. The three most common findings in a Google Cloud security review are, predictably: **broad Owner/Editor grants**, **unmanaged exported service-account keys**, and **role sprawl** — permissions granted once and never removed. This solution establishes **Cloud IAM governance**: a repeatable, least-privilege access model that is **intentional and auditable**, deployed **pilot-first** with explicit gates and a documented rollback.

> ***Vendor boundary.*** *The* ***Google Cloud Console*** *(console.cloud.google.com), the* `gcloud` ***CLI****, and* ***Cloud Identity / Google Admin*** *are authored at exact, current command / click detail from Google's public documentation. Where a capability or default is version- or preview-dependent — Workload Identity Federation provider types, org-policy constraint names, IAM Recommender / Policy Analyzer availability, IAM Conditions — it is* ***cited or flagged "verify in your organization."*** *Terraform / Cloud Foundation Toolkit is referenced at the* ***pattern level*** *with representative snippets; module internals defer to the tool's own docs. Automation is* ***Python*** *over* `gcloud` */ the Cloud Client Libraries —* ***no exported keys, no hardcoded credentials, no secrets in code.***

The Cloud IAM model — who can do what, deliberately

Google Cloud IAM answers one question — *which principal has which role on which resource* — and the answer is inherited **down the resource hierarchy** (Organization → Folder → Project → resource). Governing IAM means controlling each part of that sentence on purpose.

Principals — humans as groups, workloads as service accounts

- **Humans** are never bound individually. Access is granted to **Google Groups** sourced from the

identity system (Cloud Identity / Workspace, typically synchronized from the enterprise IdP), so joiner/mover/leaver is handled by group membership, not by editing IAM policies. One person leaving a team is a group change, not a hunt across every project.

- **Workloads** run as **service accounts**. External and federated workloads (CI/CD, on-prem, other

clouds, GitHub Actions) authenticate via **Workload Identity Federation (WIF)** — short-lived, automatically-rotated credentials — instead of a **downloaded service-account key** that never expires and cannot be revoked without breaking every consumer.

Roles — predefined and custom, never primitive

- **Primitive roles** (`roles/owner`, `roles/editor`, `roles/viewer`) are coarse legacy grants that

span thousands of permissions. Owner and Editor are effectively "can do almost anything"; they are the finding, not the fix. This solution **eliminates primitive grants** (except a tightly-audited break-glass path) in favor of scoped roles.

- **Predefined roles** are service-scoped, Google-maintained role bundles (e.g.

`roles/storage.objectViewer`, `roles/compute.instanceAdmin.v1`). **Prefer predefined roles** — they track new permissions as services evolve, so you do not have to.

- **Custom roles** are authored only when no predefined role fits — to trim a predefined role down to

the exact permissions a job needs. They carry a maintenance cost (you own the permission list as services change), so they are the exception, defined at the Organization level for reuse.

Bind at the right level of the hierarchy

A binding at the **Organization** node is inherited by every folder and project beneath it; a binding at a **project** affects only that project. Grant each role at the **highest level that is still correct and no higher** — platform-team roles high, application-team roles on their own folder or project — so access is broad only where it is genuinely meant to be. Over-broad *scope* is as much a finding as an over-broad *role*.

Conditions and guardrails

- **IAM Conditions** attach an expression (resource name prefix, request time, resource tag) to a

binding so a grant applies only in the intended context — e.g. a role that is valid only on resources tagged `env=dev`. (IAM Conditions support varies by role/resource — **verify** the target supports the condition you need.)

- **IAM-restricting org policies** cap what any principal can grant, regardless of their role:

`iam.disableServiceAccountKeyCreation` (block exported keys), `iam.allowedPolicyMemberDomains` (**Domain-Restricted Sharing** — only your domains can be granted), `iam.automaticIamGrantsForDefaultServiceAccounts` (stop default SAs receiving Editor), and `constraints/iam.serviceAccountKeyExpiryHours` where applicable. These are **guardrails**: they make the wrong grant *impossible*, not just discouraged. (Constraint availability is version-dependent — **verify** each in your organization.)

Review — access is a cadence, not an event

Least privilege decays. A recurring **access review** re-attests bindings: **IAM Recommender** and **Policy Analyzer** surface over-granted and unused roles; owners confirm or the role is trimmed. The model only stays true if it is re-checked on a schedule.

Reference design — the target end state

- **No primitive Owner/Editor** on projects except an audited break-glass group.
- **Human access** via Google Groups bound to **predefined (or trimmed custom) roles**, at the

correct hierarchy level.

- **Workload access** via service accounts, with **WIF for every external/federated caller** and

`iam.disableServiceAccountKeyCreation` **enforced** so new exported keys cannot be created.

- **Org-policy guardrails** (Domain-Restricted Sharing, key-creation disable, default-SA grant

disable) applied at the Organization node.

- **A quarterly access-review runbook** driven by Recommender / Policy Analyzer output.

Deployment model — Assess → Design → Configure → Pilot → Validate → Expand

Every change follows the category-standard staged model with explicit pass/fail gates and a documented rollback (mandatory for org-wide changes):

1. **Assess** — inventory the current IAM posture with `GCPIAM-Assess.py`: primitive-role grants,

service-account **keys**, external/federated principals, and role sprawl. Establish the baseline.

1. **Design** — define the role model (predefined-first, the short list of custom roles), the

group-to-role map, the hierarchy level for each binding, and the guardrail policy set.

1. **Configure** — author custom roles and org policies in **dry-run** (`GCPIAM-RoleDesigner.py

--dry-run, org policy dryRunSpec), scoped to a **pilot folder/project** (IAM Governance – Pilot`), not the whole organization.

1. **Pilot** — apply group-based bindings, stand up **WIF** for a pilot workload

(`GCPIAM-WIFSetup.py`), and enforce guardrails on `IAM Governance – Pilot`. **Gate:** no primitive grants, no new keys creatable, pilot workload authenticates via WIF, no legitimate access broken.

1. **Validate** — run `GCPIAM-PolicyCheck.py` and `GCPIAM-Review.py`; confirm the reference-design

assertions hold and produce the access-review report. Promote org-policy `dryRunSpec` → enforced only after clean results.

1. **Expand** — roll out folder-by-folder / project-by-project, keeping the rollback (restore prior

IAM policy from the captured baseline, set org policies back to `dryRun`) documented at each step.

What this solution delivers

• The **Solution Overview** (this document) — the Cloud IAM governance model.

• A **Requirements Guide** — organization, org-admin access, group source of truth, gcloud

toolchain, and pilot scope, each with verify / implement-if-missing.

• An **Implementation Guide** — the role model, group-based bindings at the right level, WIF to kill

exported keys, IAM-restricting org policies, and the access-review cadence; pilot then expand.

• **5 Python tools** — IAM-posture assessment (read-only), a custom-role designer/applier (dry-run),

a WIF setup helper, an access-review report generator, and an org-policy guardrail check — authentic `gcloud` / Cloud Client Library automation, read-only or `--dry-run` by default, no keys in code.

• **Testing, Troubleshooting, and Best-Practices** guides plus a deliverables summary.

Bottom line: Google Cloud access under least-privilege control — role sprawl tamed, exported keys eliminated, bindings reviewed — intentional and auditable across the organization, proven on a pilot before you scale.

2. Requirements — Google Cloud IAM & Least-Privilege Governance

This solution assumes the **shared GCP category requirements** in `global-requirements-gcp.md` are met — a **Google Cloud Organization** linked to Cloud Identity/Workspace (§1), **Organization-level administrative access** (§2), a **Cloud Billing account** (§3), the **gcloud / IaC toolchain and API enablement** (§4), a **resource-hierarchy, naming & environment design** with a **named pilot scope**

(§5), and engagement with **security / governance stakeholders** (§6). Do not re-establish those here; verify them, then confirm the **solution-specific** prerequisites below.

Each prerequisite is listed as **Description / Why required / How to verify / How to implement if missing.**

2.1 IAM administrative roles at the Organization node (not primitive Owner)

- **Description:** The operator running this solution holds the **specific** IAM-governance roles at

the Organization: `roles/resourcemanager.organizationAdmin` (see IAM policies), `roles/iam.organizationRoleAdmin` (create/manage **custom roles**), `roles/orgpolicy.policyAdmin` (set **org policies**), and `roles/iam.workloadIdentityPoolAdmin` (create **WIF** pools/providers). Recommender/Analyzer use adds `roles/recommender.iamAdmin` and `roles/policyanalyzer.activityAnalysisViewer` (name varies by edition — **verify in your organization**).

- **Why required:** Authoring custom roles, setting org-policy guardrails, and standing up Workload

Identity Federation each require a distinct admin role; the goal is to run this work **without** relying on primitive `roles/owner`.

- **How to verify:**

```
gcloud organizations get-iam-policy $ORG_ID \
  --flatten="bindings[].members" \
  --filter="bindings.members:user:you@your-domain.com" \
  --format="table(bindings.role)"
```

Confirm the roles above appear (and that you are **not** relying on `roles/owner`).

- **How to implement if missing:** Have a super-admin grant the named roles under change control

(`gcloud organizations add-iam-policy-binding $ORG_ID --member=... --role=...`). Keep the **break-glass super-admin** as a separate, MFA-protected, audited account used only when a named admin role cannot perform the action.

2.2 Google Groups as the source of truth for human access

- **Description:** Enterprise **Google Groups** (in Cloud Identity / Workspace), typically synchronized

from the enterprise IdP, that represent job functions / teams — the principals you will bind roles to. Individual-user bindings are the exception, not the rule.

- **Why required:** Group-based bindings make joiner/mover/leaver a membership change instead of an

IAM-policy edit, and keep the policy readable. Binding individuals directly is the root cause of role sprawl.

- **How to verify:** In the **Google Admin console** (admin.google.com → Directory → Groups), confirm

role-mapped groups exist (e.g. `gcp-platform-admins@`, `gcp-app-team-a-developers@`). Confirm whether membership is IdP-synchronized or managed in Google.

- **How to implement if missing:** Create the role-mapped groups (or provision them from the IdP), agree

the **group-to-role map** with team owners, and confirm ownership of each group before binding roles to it.

2.3 Workload inventory and a federation plan for external callers

• **Description:** A list of **service accounts** and the workloads that use them, flagging every

workload that today authenticates with a **downloaded service-account key** — CI/CD runners, on-prem jobs, other clouds (AWS/Azure), GitHub Actions, or partner integrations. For each, the external identity provider's issuer (OIDC issuer URL or the AWS account) that WIF will trust.

• **Why required:** Replacing exported keys with **Workload Identity Federation** requires knowing which

workloads exist, who owns them, and what external IdP each trusts — WIF trades a static key for a short-lived token brokered from that IdP.

• **How to verify:** Enumerate service-account keys with `GCPIAM-Assess.py` (or

`gcloud iam service-accounts keys list --iam-account=SA_EMAIL` per account) and identify **user-managed** keys (the exportable ones). Map each key back to a consuming workload and its owner.

• **How to implement if missing:** Build the SA→workload→owner→external-IdP map before configuring WIF;

a key with no known consumer is itself a finding — investigate and revoke.

2.4 A named pilot folder or project for IAM changes

• **Description:** A **pilot scope** — a folder or a small number of projects (`IAM Governance – Pilot`)

where the role model, group bindings, WIF, and org-policy guardrails are applied and validated **before** any Organization-node change.

• **Why required:** IAM and org-policy changes inherit **downward**; an over-broad org-node change can

lock out legitimate access instantly. Piloting on a folder contains blast radius and lets guardrails run in **dry-run** first.

• **How to verify:** Confirm the pilot folder/project exists (`gcloud resource-manager folders list

--organization=$ORG_ID`) and that you may apply IAM and org policy there.

• **How to implement if missing:** Create the pilot folder/project under the Organization and record its

ID; all Configure/Pilot steps target it by the name `IAM Governance – Pilot`.

2.5 Required APIs enabled on the administration project

• **Description:** On the project you run tooling from, enable **Cloud Resource Manager**

(`cloudresourcemanager.googleapis.com`), **IAM** (`iam.googleapis.com`), **IAM Credentials** (`iamcredentials.googleapis.com`, for WIF token exchange), **Service Usage** (`serviceusage.googleapis.com`), **Org Policy** (`orgpolicy.googleapis.com`), **Security Token Service** (`sts.googleapis.com`, for WIF), and **Recommender** (`recommender.googleapis.com`) / **Policy Analyzer** (`policyanalyzer.googleapis.com`) for the review tooling.

• **Why required:** The `gcloud` commands and the Python tools call these APIs; a disabled API returns a

`SERVICE_DISABLED` error rather than data.

• **How to verify:** `gcloud services list --enabled --project=ADMIN_PROJECT` and confirm each ID is

present.

• **How to implement if missing:**

```
gcloud services enable cloudresourcemanager.googleapis.com iam.googleapis.com \
  iamcredentials.googleapis.com serviceusage.googleapis.com orgpolicy.googleapis.com \
  sts.googleapis.com recommender.googleapis.com policyanalyzer.googleapis.com \
  --project=ADMIN_PROJECT
```

2.6 Application Default Credentials for the Python tools (no keys on disk)

• **Description:** The provided Python tools authenticate via **Application Default Credentials (ADC)** — `gcloud auth application-default login` for an operator workstation, or the **attached** service account / **WIF** on a runner. **No exported key file** is used or referenced.

• **Why required:** The whole point of this solution is to eliminate exported keys; the tooling must

practice it. ADC lets `gcloud` and the Cloud Client Libraries obtain short-lived credentials without a static key.

• **How to verify:** `gcloud auth application-default print-access-token` returns a token; confirm no

`GOOGLE_APPLICATION_CREDENTIALS` points at a downloaded key file (`echo $GOOGLE_APPLICATION_CREDENTIALS` should be empty on a workstation using user ADC).

• **How to implement if missing:** Run `gcloud auth application-default login` (workstation) or attach a

least-privilege service account / configure WIF (pipeline). Never commit or reference a key file.

Global Requirements — Google Cloud Platform (GCP) Foundation Solutions

These requirements apply to the **Google Cloud Platform (GCP)** infrastructure and zero-trust solutions in the **Google** category (landing zone, Cloud IAM, networking / VPC Service Controls, Security Command Center, FinOps, and BeyondCorp Enterprise). They are the common foundation each GCP solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every GCP solution).*** *The **Google Cloud Console** (console.cloud.google.com), the `gcloud` **CLI**, and **Cloud Identity / Google Admin** are authored at exact, current click-by-click / command detail from Google's public documentation. Where a capability, API, or default is **version- or preview-dependent** (new SCC tiers, BeyondCorp features, org-policy constraints, service-perimeter behaviors), it is **cited or flagged "verify in your organization"** — never invented. **Infrastructure-as-Code** (Terraform, the Cloud Foundation Toolkit / Terraform blueprints, Config Controller) is used at the **pattern level** — the solutions give the design, the `gcloud`/console procedure, and representative IaC, and defer provider/module internals to the tool's own docs. Automation is delivered as **Python** (Cloud Client Libraries / `gcloud` wrappers), never with*

hardcoded credentials or keys.

1. A Google Cloud Organization linked to Cloud Identity / Workspace

• **Description:** A **GCP Organization** resource (the root of the resource hierarchy), which exists once the domain's **Cloud Identity** or **Google Workspace** account is set up. Solutions build the folder/project hierarchy beneath it.

• **Why required:** Org-level policy, IAM, VPC Service Controls, and SCC all operate at or below the Organization node; without it there is no place to anchor centralized governance.

• **How to verify:** In the Cloud Console resource selector (or `gcloud organizations list`), confirm an Organization resource exists and its ID.

• **How to implement if missing:** Establish Cloud Identity/Workspace for the domain so the Organization is created, then verify domain ownership.

2. Organization-level administrative access

• **Description:** An account (ideally a **break-glass super-admin** plus a delegated **Organization Administrator** / least-privilege roles the solution names) able to set org policies, create folders/projects, and grant IAM at the Organization node.

• **Why required:** Foundational changes (resource hierarchy, org policies, IAM bindings, VPC-SC perimeters) require Organization-level authority.

• **How to verify:** Confirm the account holds `roles/resourcemanager.organizationAdmin` (and the specific roles each solution lists) via `gcloud organizations get-iam-policy`.

• **How to implement if missing:** Have a super-admin grant the roles under change control; keep break-glass accounts audited and MFA-protected.

3. A Cloud Billing account

• **Description:** An active **Cloud Billing account** linked (or linkable) to the projects the solution provisions, with a billing administrator available.

• **Why required:** Projects that host resources (networks, SCC, workloads) must be linked to billing; FinOps/budget work operates directly on the billing account.

• **How to verify:** `gcloud billing accounts list` shows an OPEN account; confirm billing-admin access.

• **How to implement if missing:** Create/associate a billing account (Google or reseller) before provisioning billable resources.

4. The gcloud / IaC toolchain and API enablement

• **Description:** A workstation or pipeline with the **Google Cloud SDK (**`gcloud`**,** `gsutil`**)**, **Python 3.10+** with the **Cloud Client Libraries**, and (where used) **Terraform**; plus the ability to **enable the required Google Cloud APIs** on target projects.

• **Why required:** Configuration, automation, and the provided Python tools run through these; most services require their API to be enabled first.

• **How to verify:** `gcloud version`, `python --version`, and `gcloud services list --enabled` on a target project.

• **How to implement if missing:** Install the SDK + libraries; enable the named APIs (`gcloud services enable …`) before running the steps.

5. A resource-hierarchy, naming & environment design

• **Description:** An agreed **folder/project structure** (e.g. by department/environment), a **naming/labeling convention**, and the **environments** in scope (prod / non-prod / shared), plus a **pilot project or folder** to validate on before org-wide rollout.

• **Why required:** IAM, org policy, networking, and cost allocation all key off the hierarchy and labels; every solution follows an **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model.

• **How to verify:** Confirm the intended hierarchy, naming convention, and a named pilot scope with the platform owners.

• **How to implement if missing:** Define the hierarchy and naming standard (this is often the first solution — the Landing Zone) before building dependent controls.

6. Security, network & governance stakeholders and policy

• **Description:** Engagement with the **security (CISO/SecOps), network, and cloud-governance / FinOps** stakeholders whose policies the configuration must satisfy (least-privilege model, allowed regions, data-residency, network egress, threat-response ownership, budget/showback rules).

• **Why required:** Org policies, IAM boundaries, perimeter design, and cost governance must align to enterprise security and financial policy, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named approvers/owners for security, networking, and cost.

• **How to implement if missing:** Identify the owners and capture the policy requirements before applying org-wide guardrails.

*The deployment model across every GCP solution is the same: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for org-wide changes. Automation is delivered as **Python** (`gcloud` / Cloud Client Libraries) — never with hardcoded credentials,

exported service-account keys committed to disk, or secrets in code.*

3. Implementation — Google Cloud IAM & Least-Privilege Governance

Follow these steps in order. They implement the **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model. Every artifact is **named** with the `<Purpose> – <Scope> – <Stage>` convention and that exact name is reused in Testing, Troubleshooting, and the scripts. All write operations are **dry-run first**; the org-wide rollout at the end has an explicit rollback.

Set working variables once:

```
export ORG_ID="123456789012"                 # your Organization ID
export ADMIN_PROJECT="iam-governance-admin"   # project you run tooling from
export PILOT_FOLDER_ID="234567890123"         # the "IAM Governance – Pilot" folder
```

Stage 1 — Assess (read-only baseline)

1.1 Capture the current IAM posture

Run the assessment tool at the Organization scope (read-only):

```
python GCPIAM-Assess.py --organization "$ORG_ID" --out iam-baseline.json
```

It reports, and writes to the baseline file:

- **Primitive-role grants** — every `roles/owner` / `roles/editor` binding, by principal and resource.
- **Service-account keys** — all **user-managed** (exportable) keys, by service account.
- **External principals** — bindings to `user:` / `serviceAccount:` outside your domains.
- **Role sprawl** — direct **user** (non-group) bindings and per-project role counts.

Keep `iam-baseline.json` — it is the **rollback reference** and the "before" side of validation.

1.2 Export the current IAM policies for rollback

For the pilot scope and any project you will touch, snapshot the exact policy:

```
gcloud projects get-iam-policy PILOT_PROJECT_ID --format=json > iam-policy-PILOT_PROJECT_ID.backup.json
gcloud resource-manager folders get-iam-policy "$PILOT_FOLDER_ID" --format=json > iam-policy-pilotfolder.b
```

> ***Gate (Assess):*** *you have a complete inventory of primitive grants, exportable keys, and external principals, plus a restorable IAM-policy snapshot for every in-scope resource. Do not proceed without the backups.*

Stage 2 — Design (author on paper, apply nothing)

2.1 Define the role model — `Role Model – Org – v1`

Produce a table (record it in your design doc) mapping each **Google Group** to the **least-privilege roles** it needs and the **hierarchy level** to bind at:

Group	Role(s)	Bind at	Why this level
`gcp-platform-admins@`	`roles/resourcemanager.folderAdmin`, `roles/iam.securityReviewer`	Organization	platform team governs all
`gcp-app-team-a-developers@`	`roles/compute.instanceAdmin.v1`, `roles/storage.objectAdmin`	Folder `App Team A`	scoped to their folder
`gcp-billing-viewers@`	`roles/billing.viewer`	Organization	read-only, org-wide
`gcp-break-glass@`	`roles/owner` (audited, IAM-Conditioned/time-bound)	Organization	emergency only

Rules: **predefined roles first**; a **custom role** only where no predefined role fits; **no** `roles/owner`/`roles/editor` except the audited break-glass group.

2.2 Design custom roles only where predefined roles do not fit

For each gap, list the exact permissions. Design (do not yet apply) with the role designer in dry-run:

```
python GCPIAM-RoleDesigner.py --organization "$ORG_ID" \
  --role-id customAppDeployer --title "Custom App Deployer – Org – v1" \
  --permissions run.services.get,run.services.update,artifactregistry.repositories.downloadArtifacts \
  --dry-run
```

Name each custom role `Custom <Function> – Org – v1`. Define custom roles at the **Organization** level so they are reusable across projects.

2.3 Design the guardrail set — `IAM Guardrails – Org – v1`

Decide which IAM-restricting org-policy constraints apply (all applied in `dryRunSpec` first):

- `constraints/iam.disableServiceAccountKeyCreation` — block new **exported** keys.
- `constraints/iam.allowedPolicyMemberDomains` — **Domain-Restricted Sharing** (only your customer IDs).
- `constraints/iam.automaticIamGrantsForDefaultServiceAccounts` — stop default SAs getting Editor.
- `constraints/iam.disableServiceAccountKeyUpload` and, where available, `constraints/iam.serviceAccountKeyExpiryHours` — **verify constraint availability in your organization.**

2.4 Plan the WIF replacement — `WIF Pool – Pilot – v1`

For each pilot workload on an exported key, record its external IdP (OIDC issuer or AWS account), the subject/attribute to map, and the target service account to impersonate.

> ***Gate (Design):*** *role-to-group map complete, custom roles minimized and dry-run-validated, guardrail set chosen, WIF plan written. No changes applied yet.*

Stage 3 — Configure (apply to the pilot scope only, dry-run guardrails)

3.1 Create the custom roles — `Custom <Function> – Org – v1`

Apply the designed custom roles at the Organization (they are definitions; they grant nothing until bound):

```
python GCPIAM-RoleDesigner.py --organization "$ORG_ID" \
  --role-id customAppDeployer --title "Custom App Deployer – Org – v1" \
  --permissions run.services.get,run.services.update,artifactregistry.repositories.downloadArtifacts \
  --apply
```

3.2 Apply group-based bindings on the pilot scope

Bind the designed groups→roles on the **pilot folder** (`IAM Governance – Pilot`) and pilot projects — **never individual users**:

```
gcloud resource-manager folders add-iam-policy-binding "$PILOT_FOLDER_ID" \
  --member="group:gcp-app-team-a-developers@your-domain.com" \
  --role="roles/compute.instanceAdmin.v1"
```

Where a grant should be context-limited, attach an **IAM Condition** (verify the role/resource supports it):

```
gcloud projects add-iam-policy-binding PILOT_PROJECT_ID \
  --member="group:gcp-app-team-a-developers@your-domain.com" \
  --role="roles/storage.objectAdmin" \
  --condition='expression=resource.name.startsWith("projects/_/buckets/appa-"),title=appa-buckets-only'
```

3.3 Stand up Workload Identity Federation — `WIF Pool – Pilot – v1`

Replace one pilot workload's exported key with WIF (example: GitHub Actions OIDC):

```
python GCPIAM-WIFSetup.py --project "$ADMIN_PROJECT" \
  --pool-id wif-pool-pilot --pool-name "WIF Pool – Pilot – v1" \
  --provider-id github --issuer-uri "https://token.actions.githubusercontent.com" \
  --attribute-mapping "google.subject=assertion.sub,attribute.repository=assertion.repository" \
  --service-account "app-a-ci@${ADMIN_PROJECT}.iam.gserviceaccount.com" \
  --allowed-attribute "attribute.repository/your-org/app-a" \
  --dry-run
```

Review the plan, then re-run with `--apply`. The tool prints the **workload-side** configuration the CI uses; the workload obtains **short-lived** credentials and the exported key is retired.

3.4 Retire the exported key (after the workload is on WIF)

Once the pilot workload authenticates via WIF and its jobs pass:

```
gcloud iam service-accounts keys list --iam-account=app-a-ci@${ADMIN_PROJECT}.iam.gserviceaccount.com
gcloud iam service-accounts keys delete KEY_ID --iam-account=app-a-ci@${ADMIN_PROJECT}.iam.gserviceaccount
```

3.5 Apply guardrails in dry-run on the pilot scope — `IAM Guardrails – Org – v1`

Apply each constraint in `dryRunSpec` on the pilot folder so violations are *logged, not enforced*:

```
python GCPIAM-PolicyCheck.py --resource "folders/$PILOT_FOLDER_ID" \
  --constraint constraints/iam.disableServiceAccountKeyCreation --set-dry-run
```

(For Domain-Restricted Sharing, supply your allowed **customer IDs**, not domains.)

> ***Gate (Configure):*** *custom roles exist, pilot bindings are **group-based**, the pilot workload runs on WIF, its exported key is deleted, and guardrails are in dry-run on the pilot folder.*

Stage 4 — Pilot (enforce on the pilot scope, observe)

4.1 Promote guardrails to enforced on the pilot folder

After a clean dry-run period (guardrail violations reviewed, none legitimate):

```
python GCPIAM-PolicyCheck.py --resource "folders/$PILOT_FOLDER_ID" \
  --constraint constraints/iam.disableServiceAccountKeyCreation --enforce
```

4.2 Confirm nothing legitimate broke

Re-run `GCPIAM-Assess.py --folder "$PILOT_FOLDER_ID"` and confirm pilot teams still have the access they need and none they should not.

> ***Gate (Pilot):*** *on* `IAM Governance – Pilot` *—* ***zero*** *primitive grants (except audited break-glass),* ***key creation blocked*** *(a test* `keys create` *is denied), pilot workload authenticates via WIF, and no legitimate access is broken.* ***Fail → rollback (Stage 6.2), fix the design, re-pilot.***

Stage 5 — Validate (prove the reference design)

5.1 Assert the reference-design invariants

```
python GCPIAM-PolicyCheck.py --resource "folders/$PILOT_FOLDER_ID" --assert-reference-design
```

Expected: no primitive Owner/Editor outside break-glass, no user-managed keys creatable, all human bindings are to `group:` principals, guardrails enforced.

5.2 Generate the access-review report — `Access Review – Pilot – Q?`

```
python GCPIAM-Review.py --organization "$ORG_ID" --scope "folders/$PILOT_FOLDER_ID" \
  --out access-review-pilot.csv
```

It pulls **IAM Recommender** (over-granted / unused roles) and **Policy Analyzer** and lists each binding for owner attestation.

> ***Gate (Validate):*** *reference-design assertion passes and the access-review report is produced and attested. Only now promote guardrails toward the Organization node.*

Stage 6 — Expand (roll out) and Rollback

6.1 Expand folder-by-folder

For each remaining folder/project, repeat **Configure** → **Pilot** → **Validate** using the same named artifacts, then apply the guardrail set at the **Organization** node in `dryRunSpec` first, review org-wide violations, and only then enforce:

```
python GCPIAM-PolicyCheck.py --resource "organizations/$ORG_ID" \
  --constraint constraints/iam.disableServiceAccountKeyCreation --set-dry-run   # review first
python GCPIAM-PolicyCheck.py --resource "organizations/$ORG_ID" \
  --constraint constraints/iam.disableServiceAccountKeyCreation --enforce       # after clean review
```

6.2 Rollback (documented, per scope)

If any stage fails, restore the captured state:

```
# Restore IAM policy from the Stage-1.2 backup
gcloud projects set-iam-policy PILOT_PROJECT_ID iam-policy-PILOT_PROJECT_ID.backup.json
gcloud resource-manager folders set-iam-policy "$PILOT_FOLDER_ID" iam-policy-pilotfolder.backup.json
# Set an enforced guardrail back to dry-run (non-blocking) while you investigate
python GCPIAM-PolicyCheck.py --resource "folders/$PILOT_FOLDER_ID" \
  --constraint constraints/iam.disableServiceAccountKeyCreation --set-dry-run
```

A guardrail regression is non-destructive (dry-run keeps logging); an IAM regression is fixed by re-applying the backup policy. WIF setup is additive — retire the WIF provider only after the workload is moved back, and never re-create an exported key to "fix" a WIF issue.

5. Testing & Validation — Google Cloud IAM & Least-Privilege Governance

Run these after Configure and again after each Expand step. Each test states the procedure and the **expected pass/fail**. Tests reuse the named artifacts from the Implementation Guide (`IAM Governance – Pilot`, `IAM Guardrails – Org – v1`, `WIF Pool – Pilot – v1`).

Test 1 — No primitive Owner/Editor except audited break-glass

- **Procedure:**

```
python GCPIAM-Assess.py --folder "$PILOT_FOLDER_ID" --only primitive-roles
```

- **Pass:** the only `roles/owner`/`roles/editor` binding reported is `group:gcp-break-glass@` (and it

is time-bound / IAM-Conditioned). No user or app team holds a primitive role.

- **Fail:** any other primitive grant appears → remove it, re-bind the least-privilege role from

`Role Model – Org – v1`.

Test 2 — Human bindings are group-based, not individual

- **Procedure:**

```
python GCPIAM-Assess.py --folder "$PILOT_FOLDER_ID" --only user-bindings
```

- **Pass:** zero direct `user:` bindings (all human access is via `group:`), except any explicitly

documented exception.

- **Fail:** a `user:` binding appears → move the user into the correct Google Group and remove the direct

binding.

Test 3 — New service-account keys cannot be created (guardrail enforced)

- **Procedure:** on a pilot service account, attempt to create a key:

```
gcloud iam service-accounts keys create /dev/null \
  --iam-account=app-a-ci@${ADMIN_PROJECT}.iam.gserviceaccount.com
```

- **Pass:** the request is **denied** by org policy

(`constraints/iam.disableServiceAccountKeyCreation` on `IAM Guardrails – Org – v1`) with a `FAILED_PRECONDITION` / policy-violation error.

- **Fail:** a key is created → the constraint is still in `dryRunSpec`; promote it to enforced

(Implementation 4.1) and re-test.

Test 4 — No exportable (user-managed) keys remain on migrated accounts

- **Procedure:**

```
gcloud iam service-accounts keys list \
  --iam-account=app-a-ci@${ADMIN_PROJECT}.iam.gserviceaccount.com \
  --managed-by=user
```

- **Pass:** the list is empty (only Google-managed keys remain, which are not exportable) for every

workload moved to `WIF Pool – Pilot – v1`.

- **Fail:** a user-managed key is listed → confirm the workload is on WIF, then delete the key.

Test 5 — The pilot workload authenticates via Workload Identity Federation

- **Procedure:** trigger the pilot CI/workload job configured against `WIF Pool – Pilot – v1` and observe

it obtain credentials without a key file. Confirm the exchange in logs:

```
gcloud logging read 'protoPayload.serviceName="sts.googleapis.com"' --limit=5 --project="$ADMIN_PROJECT"
```

- **Pass:** the workload runs successfully and STS token-exchange events appear — proving short-lived,

federated credentials, no static key.

- **Fail:** the job cannot authenticate → see Troubleshooting T3 (attribute-mapping / audience mismatch).

Test 6 — Domain-Restricted Sharing blocks external principals

- **Procedure:** attempt to grant a role to an out-of-domain principal on the pilot scope:

```
gcloud projects add-iam-policy-binding PILOT_PROJECT_ID \
  --member="user:someone@example.com" --role="roles/viewer"
```

- **Pass:** denied by `constraints/iam.allowedPolicyMemberDomains` — only your customer IDs may be

granted.

- **Fail:** the grant succeeds → the constraint is not enforced or the customer-ID allow-list is wrong;

re-apply from `IAM Guardrails – Org – v1`.

Test 7 — Reference-design assertion passes

- **Procedure:**

```
python GCPIAM-PolicyCheck.py --resource "folders/$PILOT_FOLDER_ID" --assert-reference-design
```

- **Pass:** the tool exits `0` — no primitive grants (outside break-glass), no creatable user-managed

keys, all human bindings group-based, guardrails enforced.

- **Fail:** it exits non-zero and lists the violated invariant → fix that specific item and re-run.

Test 8 — Access-review report generates and reconciles

- **Procedure:**

```
python GCPIAM-Review.py --organization "$ORG_ID" --scope "folders/$PILOT_FOLDER_ID" --out review.csv
```

- **Pass:** `review.csv` lists every binding with its principal, role, and any IAM Recommender

"over-granted / unused" flag, ready for owner attestation.

- **Fail:** empty or errored output → check Recommender/Policy Analyzer APIs are enabled (Requirement

2.5) and the operator holds the viewer roles (Requirement 2.1).

Overall pass criteria (promotion gate)

Promote from pilot to expand only when **Tests 1–8 all pass** on `IAM Governance – Pilot`: no primitive grants outside break-glass, group-based human access, exported keys eliminated and un-creatable, the pilot workload on WIF, Domain-Restricted Sharing enforced, the reference-design assertion green, and the access-review report attested.

6. Troubleshooting — Google Cloud IAM & Least-Privilege Governance

Common failure modes when standing up the IAM governance model, and how to resolve them. Each maps back to a step or test in the Implementation and Testing guides and reuses the named artifacts (`IAM Governance – Pilot`, `IAM Guardrails – Org – v1`, `WIF Pool – Pilot – v1`).

T1 — A team lost access after tightening bindings ("permission denied" for legitimate work)

- **Symptom:** After replacing primitive grants with least-privilege roles, a pilot team gets

`PERMISSION_DENIED` on a task they should be able to do.

- **Cause:** The predefined/custom role in `Role Model – Org – v1` is missing a permission the workflow

needs, or the binding is at too *low* a level (project) when the resource lives in a sibling project.

- **Resolution:** Identify the exact missing permission from the error, confirm which predefined role

contains it (`gcloud iam roles describe roles/<role>`), and either add that predefined role or extend the custom role via `GCPIAM-RoleDesigner.py --apply`. If the scope is wrong, re-bind at the correct folder level. If access is broken for many users, **roll back** the IAM policy from the Stage-1.2 backup (`gcloud ... set-iam-policy ...backup.json`) and re-derive the role.

T2 — Org-policy guardrail is not blocking (a key still gets created)

- **Symptom:** Test 3 fails — `keys create` succeeds even though

`constraints/iam.disableServiceAccountKeyCreation` is "applied."

- **Cause:** The constraint is still in `dryRunSpec` (logging, not enforcing), or it is set on a

sibling scope that does not cover the target project, or a higher scope's policy overrides it.

• **Resolution:** Confirm the effective policy on the *project* that hosts the SA:

`python GCPIAM-PolicyCheck.py --resource projects/PID --constraint constraints/iam.disableServiceAccountKeyCreation`. Promote dry-run → enforced at the correct scope (`... --resource folders/$PILOT_FOLDER_ID --constraint ... --enforce`), remembering org policy is inherited **downward** — enforce at a level that is an ancestor of the target.

T3 — WIF workload cannot authenticate (STS token exchange fails)

• **Symptom:** The pilot workload on `WIF Pool – Pilot – v1` fails with an `invalid_target`,

`unauthorized_client`, or "subject does not match" error; no STS event, or a rejected one.

• **Cause:** The **audience** in the workload config does not match the provider resource, the

attribute-mapping or **attribute-condition** does not match the token's claims, or the `principalSet://` member does not match the mapped attribute value.

• **Resolution:** Re-run `GCPIAM-WIFSetup.py` **without** `--apply` to reprint the exact audience and

workload-side config, and compare against what the workload sends. Confirm the `--allowed-attribute` (e.g. `attribute.repository/org/repo`) exactly matches the external token's claim, and that the OIDC `--issuer-uri` is the IdP's real issuer (**verify against the external IdP's docs**). Check the STS events: `gcloud logging read 'protoPayload.serviceName="sts.googleapis.com"'`.

T4 — Access-review report is empty or missing Recommender flags

• **Symptom:** `GCPIAM-Review.py` produces bindings but no `recommender_flag` values, or errors on the

Recommender call.

• **Cause:** The **Recommender API** (`recommender.googleapis.com`) is not enabled, the caller lacks

`roles/recommender.iamViewer`, or the recommendations have not yet been generated (Recommender needs observation time and applies at **project** scope). Availability can be **edition-dependent — verify in your organization.**

• **Resolution:** Enable the API (Requirement 2.5), grant the viewer role (Requirement 2.1), and run the

review at a **project** scope. Recommendations populate after Google has observed usage; a brand-new project will have none yet — re-run after the observation window.

T5 — Custom role rejects a permission ("permission is not supported")

• **Symptom:** `GCPIAM-RoleDesigner.py --apply` fails because a listed permission cannot be used in a

custom role.

• **Cause:** Some permissions are **not grantable in custom roles** (they are only available through

specific predefined roles), or the permission name is mistyped / has been renamed.

• **Resolution:** Remove the unsupported permission and, if the capability is genuinely needed, grant the

predefined role that carries it instead of forcing it into a custom role. Verify exact permission names with `gcloud iam list-testable-permissions //cloudresourcemanager.googleapis.com/projects/PID`.

T6 — Domain-Restricted Sharing blocks a legitimate external principal

- **Symptom:** After enforcing `constraints/iam.allowedPolicyMemberDomains`, a needed partner /

cross-org service account can no longer be granted a role.

- **Cause:** The allow-list holds only your **customer IDs**; the external principal's customer ID (or

the special value for federated identities) is not included.

- **Resolution:** Do **not** disable the guardrail. Add the specific, approved external **customer ID** to

the `allowedPolicyMemberDomains` allow-list under change control (or provision the external party via a WIF/federated principal instead of a direct cross-domain grant), then re-test the grant.

T7 — "You need permission" when setting org policy or creating custom roles

- **Symptom:** `gcloud org-policies ...` or `gcloud iam roles create ...` returns `PERMISSION_DENIED` for

the operator.

- **Cause:** The operator holds `roles/owner` expectations but not the **specific** admin role — setting

org policy needs `roles/orgpolicy.policyAdmin`; custom roles need `roles/iam.organizationRoleAdmin`; WIF needs `roles/iam.workloadIdentityPoolAdmin` (Requirement 2.1).

- **Resolution:** Have a super-admin grant the **named** role at the Organization (not primitive Owner).

This is by design — the solution deliberately runs without relying on Owner. Use the audited break-glass path only if a named role genuinely cannot perform the action.

T8 — Locked out at the Organization node after an org-wide change

- **Symptom:** An org-node IAM or org-policy change removed the operator's own access, or broke access

broadly.

- **Cause:** An over-broad org-node change applied without a dry-run window — inheritance pushed it to

every folder/project instantly.

- **Resolution:** Use the **break-glass super-admin** to restore the org-node IAM policy from the backup

and set the offending org policy back to `dryRunSpec` (rollback, Implementation 6.2). Then re-apply the change **on the pilot scope first** with a dry-run window, per the deployment model — never enforce a new org-node change without piloting it.

7. Best Practices — Google Cloud IAM & Least-Privilege Governance

Durable practices that keep the least-privilege model true after the project ends. These are the habits that stop role sprawl and exported keys from creeping back.

Roles

- **Predefined-first, custom-by-exception.** Reach for a predefined role every time; author a custom

role only when no predefined role fits, and keep the custom-role count small — each one is a permission list you now own as services evolve.

- **Never grant primitive roles.** `roles/owner` and `roles/editor` are the finding, not the fix.

Keep exactly one audited, time-bound, IAM-Conditioned **break-glass** path and treat any other primitive grant as an incident.

- **Bind at the highest level that is still correct — and no higher.** Platform roles at the

Organization/folder, application roles on the app's own folder/project. Over-broad *scope* is as much a finding as an over-broad *role*.

- **Use IAM Conditions to narrow, not to compensate.** A condition tightens an already-appropriate

role (resource prefix, tag, time); it is not a way to make a too-powerful role "safe."

Principals

- **Humans as groups, always.** Bind roles to Google Groups sourced from the IdP so joiner/mover/leaver

is a membership change. Direct `user:` bindings are the seed of sprawl — allow them only as documented, time-boxed exceptions.

- **One workload, one service account, least privilege.** Do not share a service account across

workloads, and do not reuse a high-privilege SA for a low-privilege job. Disable the **default** service accounts' automatic Editor grant (`constraints/iam.automaticIamGrantsForDefaultServiceAccounts`).

- **Prefer impersonation over keys for human-to-SA.** When a human needs to act as a service account,

grant `roles/iam.serviceAccountTokenCreator` and impersonate (`--impersonate-service-account`) — a short-lived token, not a downloaded key.

Kill exported keys

- **Enforce** `iam.disableServiceAccountKeyCreation` **org-wide** once migrated — this is the single most

effective guardrail. A key that cannot be created cannot be leaked.

• **Workload Identity Federation for every external/federated caller.** CI/CD, other clouds, on-prem, partner integrations — all get short-lived federated tokens, never a static key. Map the **narrowest** external attribute (specific repo, specific branch, specific AWS role) that identifies the workload.

• **Treat any surviving user-managed key as a live secret** — rotate on a short cadence until it is eliminated, and alert on `keys create` in Cloud Audit Logs.

Guardrails

• **Domain-Restricted Sharing on.** `iam.allowedPolicyMemberDomains` scoped to your customer IDs stops accidental external grants at the door.

• **Dry-run every org-policy change first.** Apply constraints in `dryRunSpec`, review the logged violations, then enforce. Never enforce a new constraint at the Organization node without a dry-run window.

• **Version and name your policy set** (`IAM Guardrails – Org – v1`) and manage it as code (Terraform / Cloud Foundation Toolkit) so guardrails are reviewable and reproducible.

Review as a cadence

• **Attest bindings quarterly.** Drive the review from **IAM Recommender** (unused / over-granted roles) and **Policy Analyzer**; every binding is confirmed by an owner or trimmed. An un-reviewed binding is an expired binding.

• **Act on Recommender.** Over time, adopt its safe least-privilege recommendations rather than letting them accumulate — the whole point is to *converge* on least privilege, not just measure the gap.

• **Log and alert on IAM changes.** Route `SetIamPolicy`, `keys create`, and org-policy changes from Cloud Audit Logs to your SIEM so drift is visible between reviews.

Tooling discipline

• **Everything read-only or dry-run by default.** The provided tools apply only with an explicit `--apply`; keep that discipline in anything you add.

• **ADC only — no key files.** Run tooling under user ADC, an attached service account, or WIF. Never set `GOOGLE_APPLICATION_CREDENTIALS` to a downloaded key or commit one to a repo.

• **Manage IAM as code where you can.** Bindings and policies expressed in Terraform / CFT are reviewable, diffable, and roll back cleanly — far safer than click-ops at the Organization node.

8. Deliverables Summary — Google Cloud IAM & Least-Privilege Governance

Everything included in this solution, on one page.

Documentation

- **01 — Solution Overview.** The Cloud IAM governance model: principals (groups + service accounts), predefined vs custom roles, binding at the right hierarchy level, Workload Identity Federation, IAM-restricting org policies, and the review cadence — plus the reference design and deployment model.
- **02 — Requirements.** Solution-specific prerequisites (org-admin IAM roles, Google Groups as source of truth, workload/federation inventory, pilot scope, required APIs, ADC), each with verify / implement-if-missing — building on the shared `global-requirements-gcp.md`.
- **03 — Implementation.** Exact ordered steps: Assess → Design → Configure → Pilot → Validate → Expand, every artifact named (`IAM Governance – Pilot`, `IAM Guardrails – Org – v1`, `WIF Pool – Pilot – v1`), with pass/fail gates and a documented rollback.
- **05 — Testing.** Eight validation tests with expected pass/fail (no primitive grants, group-based access, keys un-creatable, exported keys gone, workload on WIF, Domain-Restricted Sharing, reference assertion, access-review report).
- **06 — Troubleshooting.** Common failure modes and resolutions (locked-out access, guardrail not enforcing, WIF token exchange, Recommender empty, custom-role permission errors, key still creatable).
- **07 — Best Practices.** Durable practices for roles, principals, killing exported keys, guardrails, the review cadence, and tooling discipline.
- **Executive Summary.** The outcome, approach, and why it is safe and governed, for decision-makers.

The 5 Python tools (`gcloud` / Cloud Client Libraries; read-only or `--dry-run` by default)

1. `GCPIAM-Assess.py` — *IAM-posture assessment (read-only).* Inventories primitive-role grants, user-managed (exportable) service-account keys, external principals, and role sprawl across the Organization/folder/project; writes the baseline used for rollback and validation.
1. `GCPIAM-RoleDesigner.py` — *Custom-role designer/applier (dry-run default).* Validates a proposed custom role's permissions and prints the plan; creates/updates the Organization-level custom role only with `--apply`.
1. `GCPIAM-WIFSetup.py` — *Workload Identity Federation setup helper (dry-run default).* Plans and (with `--apply`) creates a WIF pool + OIDC/AWS provider and the SA impersonation binding, then prints the workload-side config — so exported keys can be retired.
1. `GCPIAM-Review.py` — *Access-review report generator (read-only).* Pulls IAM Recommender (unused / over-granted roles) and Policy Analyzer, and emits a per-binding CSV for owner attestation.

1. `GCPIAM-PolicyCheck.py` — *Org-policy guardrail check & reference-design assertion (dry-run default).* Reads/sets IAM-restricting org-policy constraints (dry-run → enforce) and asserts the reference-design invariants hold on a scope.

Safety: every tool authenticates via **Application Default Credentials** (no key files), is **read-only or** `--dry-run` **unless** `--apply` is passed, and contains **no hardcoded credentials, keys, or secrets**.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

GCPIAM-Assess.py

GCPIAM-PolicyCheck.py

GCPIAM-Review.py

GCPIAM-RoleDesigner.py

GCPIAM-WIFSetup.py